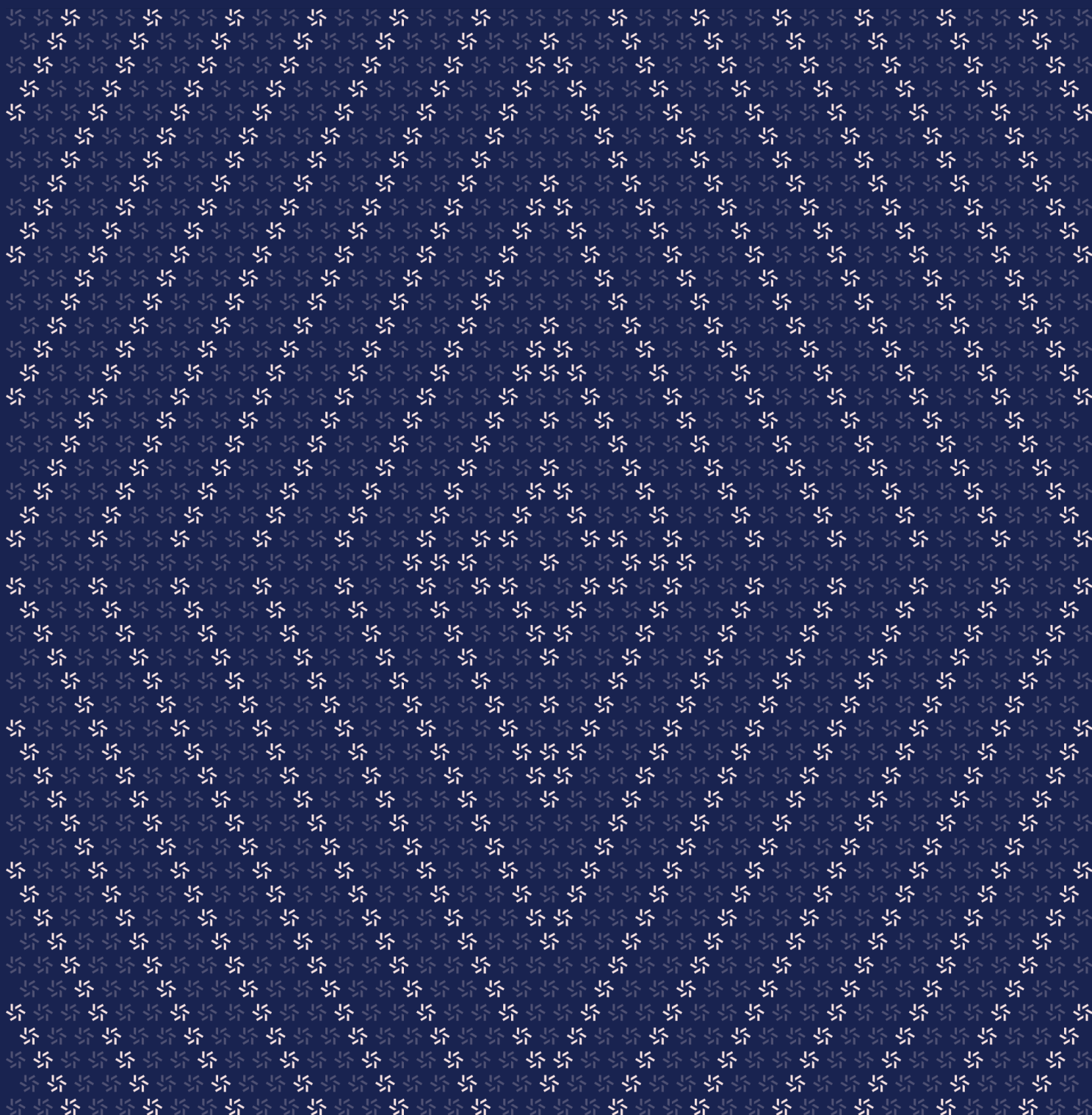


March 5, 2026

IBC Solidity Contracts

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About IBC Solidity Contracts	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. AttestationLightClient lacks cross-client domain separation	11
3.2. Empty revert data treated as being out of gas in <code>recvPacket()</code>	13
3.3. The <code>sendPacket</code> function does not verify light-client status	15
3.4. Insufficient test coverage	17
4. Discussion	18
4.1. Reverting source callbacks can block acknowledgment and time-out finalization	19
4.2. Single-key trust dependency created by <code>minRequiredSigs = 1</code>	19

5.	Threat Model	19
5.1.	Module: AttestationLightClient.sol	20
5.2.	Module: ICS20Transfer.sol	26
5.3.	Module: ICS26Router.sol	41
5.4.	Module: ICS27Account.sol	52
5.5.	Module: ICS27GMP.sol	57
5.6.	Module: IFTAccessManaged.sol	64
5.7.	Module: IFTBaseUpgradeable.sol	65
5.8.	Module: IFTOwnable.sol	73

6.	Assessment Results	78
6.1.	Disclaimer	79

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Cosmos Labs from March 3rd to March 9th, 2026. During this engagement, Zellic reviewed IBC Solidity contracts' code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the custom ICS27Account implementation secure?
 - Is the multi-sig implementation in attestation LC secure?
 - Are access controls implemented effectively to prevent unauthorized operations?
 - Are there any vulnerabilities that could result in the loss of user funds?
 - Are there any vulnerabilities that could result in denial of service?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

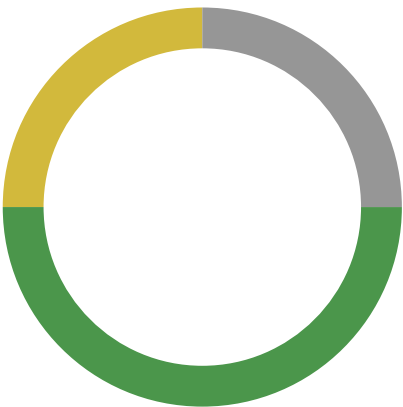
1.4. Results

During our assessment on the scoped IBC Solidity contracts, we discovered four findings. No critical issues were found. Two findings were of medium impact and two were of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Cosmos Labs in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	2
Informational	1



2. Introduction

2.1. About IBC Solidity Contracts

Cosmos Labs contributed the following description of IBC Solidity contracts:

This is an implementation of IBC v2 in Solidity.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and

Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

IBC Solidity Contracts

Type	Solidity
Platform	EVM-compatible
Target	solidity-ibc-eureka
Repository	https://github.com/cosmos/solidity-ibc-eureka
Version	9798e644ed8624d081afe417eb55b401018cd38d
Programs	ICS27GMP.sol ICS20Transfer.sol ICS26Router.sol utils/IFTBaseUpgradeable.sol utils/IFTOwnable.sol utils/IFTAccessManaged.sol utils/EVMIFTSendCallConstructor.sol utils/CosmosIFTSendCallConstructor.sol utils/ICS27Lib.sol utils/IBCSenderCallbacksLib.sol utils/IBCCallbackReceiver.sol utils/ICS27Account.sol light-clients/attestation/**

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.4 person-weeks. The assessment was conducted by two consultants over the course of one calendar week.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Hojung Han
✈ Engineer
hojung@zellic.io ↗

Jisub Kim
✈ Engineer
jisub@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 3, 2026 Kick-off call

March 3, 2026 Start of primary review period

March 9, 2026 End of primary review period

3. Detailed Findings

3.1. AttestationLightClient lacks cross-client domain separation

Target	AttestationLightClient		
Category	Cryptography	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The AttestationLightClient contract separates state and packet attestations using a one-byte type tag (0x01 for state, 0x02 for packets) prepended before hashing:

```
function _taggedSigningInput(bytes memory data, bytes1 typeTag)
    private pure returns (bytes32) {
    return sha256(abi.encodePacked(typeTag, sha256(data)));
}
```

However, the signed payload does not include any client-specific or chain-specific domain separator. The StateAttestation struct contains only height and timestamp, while PacketAttestation contains only height and packets. Neither struct binds the attestation to a particular client identifier, chain identifier, or contract address.

As a result, if the same attester key set is reused across multiple AttestationLightClient deployments, a valid attestation produced for one client can be replayed against another client of the same type, provided the attester set and quorum threshold are compatible. While the type tag prevents cross-type replay (state attestations cannot be substituted for packet attestations), same-type cross-client replay remains feasible.

The design documentation explicitly acknowledges this replay class and recommends stronger domain separation.

Impact

If attester keys are shared across deployments, a foreign attestation could be accepted by a client for which it was not intended. In IBC integrations, this could lead to unauthorized state updates or packet processing. The practical exploitability depends on deployment policy — if attester keys are unique per client instance, the risk reduces to operator policy failure.

Recommendations

Consider adding explicit domain separation to the signed payload, such as the client identifier, chain identifier, or contract address.

In the interim, consider documenting and operationally enforcing that attester keys must be unique per light-client instance and must never be reused across chains, clients, or environments. Key rotation procedures should preserve this non-overlap property.

Remediation

This issue has been acknowledged by Cosmos Labs.

3.2. Empty revert data treated as being out of gas in `recvPacket()`

Target	ICS26Router.sol		
Category	Integration Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

In the `recvPacket()` function, the catch block that handles callback failures requires nonempty revert data:

```

} catch (bytes memory reason) {
    require(reason.length != 0, IBCFailedCallback());
    emit IBCAppRecvPacketCallbackError(reason);
    acks[0] = ICS24Host.UNIVERSAL_ERROR_ACK;
}

```

This check was introduced to detect out-of-gas (OOG) conditions, which produce empty revert data. However, a callback that fails via a bare `revert()` (without a message or custom error) also produces empty revert data and is therefore indistinguishable from an OOG failure. In such cases, the entire `recvPacket()` transaction reverts with `IBCFailedCallback()` instead of committing with a universal error acknowledgment.

While the currently shipped IBCApp implementations do not use bare `revert()` in their `onRecvPacket()` paths, the ICS26Router permits permissionless registration of new applications. Third-party or future implementations that revert with empty data would trigger this behavior.

Impact

An IBCApp whose `onRecvPacket()` callback reverts with empty data causes the entire `recvPacket()` transaction to revert rather than producing a universal error acknowledgment. This deviates from the expected IBC error handling and could block packet processing for such applications.

Recommendations

Consider documenting as an integration requirement that all `onRecvPacket()` implementations must revert with nonempty revert data (e.g., a custom error or a reason string) to ensure correct error handling.

Remediation

This issue has been acknowledged by Cosmos Labs.

3.3. The sendPacket function does not verify light-client status

Target	ICS26Router.sol		
Category	Protocol Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

NOTE: This finding had also been independently identified by Cosmos Labs prior to our review.

The `sendPacket()` function in `ICS26Router` does not verify whether the referenced light client is in a frozen or expired state before accepting a packet for sending. In comparison, the reference `ibc-go` implementation checks the client status before allowing a packet to be sent.

When a packet is sent through `ICS20Transfer`, the sender's tokens are escrowed. If the light client is frozen or expired at the time of sending, the corresponding acknowledgment and time-out flows cannot execute because they depend on proof verification against the light client. This could result in user funds remaining locked until the light client is restored.

Impact

Users who send packets while the light client is frozen or expired have their tokens escrowed with no immediate mechanism to recover them. The funds remain locked until the light client is reenabled via migration. The practical likelihood is reduced by the expectation that front-end interfaces would disable transfers when a light client is inactive.

Recommendations

Consider documenting this behavior as a known limitation and ensuring that front-end interfaces check the light-client status before initiating transfers. If gas overhead is acceptable for the deployed light-client implementations, consider adding a status check in `sendPacket()` to reject packets when the client is not active.

Remediation

This issue has been acknowledged by Cosmos Labs. The Cosmos Labs team decided not to address it since:

1. SP1 light client cannot always prove that it expired on chain.
2. We can mitigate this by disabling transfers in the front-end.
3. A frozen light client must be re-enabled anyway (via migration), which would unlock the user's funds.
4. This check can cost a lot of extra gas depending on the light client.

3.4. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts, and all functions — not just surface-level functions. It is important to test the invariants required for ensuring security and also verify any mathematical properties from the project's specification. Additionally, testing cross-chain function calls and transfers is recommended to ensure the desired functionality.

Unit test cases that are missing include (but are not necessarily limited to) those documented in section [5.7](#).

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits

are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend building a rigorous test suite that includes all contracts to ensure that the system operates securely and as intended.

Remediation

This issue has been acknowledged by Cosmos Labs.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Reverting source callbacks can block acknowledgment and time-out finalization

Unlike `recvPacket`, `ackPacket` and `timeoutPacket` invoke the source-application callback directly, without a try-catch boundary. When the source callback reverts, the full transaction reverts, and the earlier packet-commitment deletion is rolled back. If the callback reverts deterministically, acknowledgment/time-out processing can be repeatedly blocked. Under IFT semantics, this can prevent burned transfers from being finalized or refunded, leaving funds in pending recovery.

This is not an immediately exploitable bug as of the time of writing. It is a design assumption that source applications will not revert during acknowledgment/time-out callbacks, with direct liveness impact if that assumption fails.

In deployed systems, ICS20, ICS27, and IFT integrations still expose revert-capable paths (e.g., paused states, decoding failures, sender callback failures, or invalid ICS27 address configuration). These conditions are often remediable through upgrades or configuration corrections, so the issue is framed as a design/liveness risk rather than a direct exploit path. Source-application integrators and operators should explicitly account for this assumption during deployment, monitoring, and upgrade planning.

4.2. Single-key trust dependency created by `minRequiredSigs = 1`

When `minRequiredSigs` is 1, the light client's trust assumption collapses to a single attester key. This weakens the Byzantine fault tolerance expected from threshold signing and creates a single point of failure for system trust. If that key is compromised, leaked, or misused, an attacker can submit arbitrary state updates. They can also submit conflicting updates at the same height and push the client into a frozen state, degrading verification availability and protocol reliability.

Accordingly, `minRequiredSigs = 1` should be documented as a lower-security configuration, not a production default. Production environments should enforce a threshold of at least two signatures — preferably higher. Deployment guidance should define key-management controls, including HSM-backed custody, strict access control, regular key rotation, and incident-response procedures. A deployment checklist should also prevent accidental single-signature dependency.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: AttestationLightClient.sol

Function: `updateClient(bytes calldata updateMsg)`

This function updates the light-client state by processing an attestation proof containing a new (height, timestamp) pair signed by the attester set. If a height already has a stored timestamp that differs, the client is frozen (misbehavior). If the timestamp matches, it is a no-op. Otherwise, the new consensus state is stored and the latest height is updated if necessary.

Inputs

- `updateMsg` (decoded as `AttestationProof`)
 - **Control:** Proof-submitter-controlled (relay or permissionless depending on role configuration).
 - **Constraints:** `attestationData` must ABI-decode as `StateAttestation` with `height > 0` and `timestamp > 0`; signatures must be nonempty, the count must be greater than or equal to `minRequiredSigs`, each 65 bytes, and each recovering to a unique known attester.
 - **Impact:** Updates the trusted consensus state. Malicious updates are prevented by requiring M-of-N attester signatures. Conflicting updates freeze the client.

Branches and code coverage

Intended branches

- New height/timestamp — stored and `latestHeight` is updated if greater. Returns `Update`.
 - ☒ Test coverage
- Height already exists with the same timestamp — returns `NoOp`.
 - ☒ Test coverage
- Height already exists with a different timestamp — freezes client and returns `Misbehaviour`.
 - ☒ Test coverage

- New height is less than or equal to the latest height — stored but latestHeight is not updated.

☒ Test coverage

Negative behavior

- Reverts with FrozenClientState if the client is frozen.
 - ☐ Negative test
- Reverts with AccessControlUnauthorizedAccount if the caller lacks PROOF_SUBMITTER_ROLE (unless open).
 - ☒ Negative test
- Reverts with EmptySignatures if the signatures array is empty.
 - ☐ Negative test
- Reverts with ThresholdNotMet if the signature count is less than minRequiredSigs.
 - ☒ Negative test
- Reverts with InvalidSignatureLength if any signature is not 65 bytes.
 - ☐ Negative test
- Reverts with SignatureInvalid if any signature recovers to address(0).
 - ☐ Negative test
- Reverts with UnknownSigner if any signer is not in the attester set.
 - ☒ Negative test
- Reverts with DuplicateSigner if there are duplicate signers.
 - ☒ Negative test
- Reverts with InvalidState if height or timestamp is zero.
 - ☐ Negative test
- Rejects cross-domain replay (packet signatures rejected as state signatures).
 - ☒ Negative test

Function call analysis

- `_taggedSigningInput(proof.attestationData, ATTESTATION_TYPE_STATE)`
 - **What is controllable?** attestationData.
 - **If the return value is controllable, how is it used and how can it go wrong?** Produces the digest `sha256(0x01 || sha256(data))`. Domain-separates state attestations from packet attestations. The caller cannot forge a valid digest without attester private keys.

- **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `_verifySignaturesThreshold(digest, proof.signatures)`
 - **What is controllable?** digest and signatures.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value. Reverts if verification fails.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Uses $O(n^2)$ duplicate check loop — acceptable for small attester sets but could be gas expensive for large sets. Uses OpenZeppelin ECDSA.`recover`, which handles signature malleability (only accepts low-s values).

Function: `verifyMembership(ILightClientMsgs.MsgVerifyMembership call-data msg_)`

This function verifies that a specific packet commitment exists on the counterparty chain at a given height. Although this is a view function, it is a critical security function in the IBC proof-verification path called by `ICS26Router.recvPacket` and `ackPacket`.

Inputs

- `msg_.value`
 - **Control:** ICS26Router-controlled (computed from packet data).
 - **Constraints:** Must be nonempty — decoded as `bytes32`.
 - **Impact:** The expected packet commitment value to verify.
- `msg_.path`
 - **Control:** ICS26Router-controlled (computed from packet data).
 - **Constraints:** Must have exactly one element.
 - **Impact:** The IBC commitment path, hashed for comparison against attested packets.
- `msg_.proofHeight.revisionHeight`
 - **Control:** Relayer-controlled.
 - **Constraints:** Must have a stored consensus timestamp.
 - **Impact:** Links the proof to a specific trusted height.
- `msg_.proof` (decoded as `AttestationProof`)
 - **Control:** Relayer-controlled.
 - **Constraints:** `attestationData` must be decoded as `PacketAttestation` with height matching `proofHeight`, `packets` must be nonempty, must contain matching (path, commitment), and signatures must meet threshold.
 - **Impact:** Proves the packet commitment was attested by the quorum.

Branches and code coverage

Intended branches

- Matching packet found in attested list — returns consensus timestamp.

☒ Test coverage

Negative behavior

- Reverts with `FrozenClientState` if the client is frozen.

☐ Negative test

- Reverts with `EmptyValue` if the value is empty.

☒ Negative test

- Reverts with `InvalidPathLength` if path length does not equal 1.

☐ Negative test

- Reverts with `ConsensusTimestampNotFound` if there is no timestamp at proof height.

☒ Negative test

- Reverts with signature errors if signature verification fails.

☐ Negative test

- Reverts with `HeightMismatch` if the attested height does not equal proof height.

☒ Negative test

- Reverts with `EmptyPackets` if the packets array is empty.

☐ Negative test

- Reverts with `NotMember` if no matching packet is found.

☒ Negative test

- Rejects cross-domain replay (state signatures rejected as packet signatures).

☒ Negative test

Function call analysis

- `_taggedSigningInput(proof.attestationData, ATTESTATION_TYPE_PACKET)`
 - **What is controllable?** `attestationData`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Produces the digest `sha256(0x02 || sha256(data))`. Domain tag `0x02` prevents cross-type replay from state attestations.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `_verifySignaturesThreshold(digest, proof.signatures)`

- **What is controllable?** signatures.
- **If the return value is controllable, how is it used and how can it go wrong?**
N/A — reverts on failure.
- **What happens if it reverts, reenters or does other unusual control flow?**
Same $O(n^2)$ pattern as in `updateClient`.

Function: `verifyNonMembership(ILightClientMsgs.MsgVerifyNonMembership calldata msg_)`

This function verifies that a specific packet receipt does *not* exist on the counterparty chain at a given height. It is used by `ICS26Router.timeoutPacket` to prove a packet was not received. Although this is a view function, it is a critical security function.

Inputs

- `msg_.path`
 - **Control:** ICS26Router-controlled (computed from packet data).
 - **Constraints:** Must have exactly one element.
 - **Impact:** The IBC receipt path to check for nonmembership.
- `msg_.proofHeight.revisionHeight`
 - **Control:** Relay-layer-controlled.
 - **Constraints:** Must have a stored consensus timestamp.
 - **Impact:** Links the proof to a trusted height.
- `msg_.proof` (decoded as `AttestationProof`)
 - **Control:** Relay-layer-controlled.
 - **Constraints:** `attestationData` must be decoded as `PacketAttestation` with height matching `proofHeight`, packets must be nonempty, must contain the path with `commitment == bytes32(0)`, and signatures must meet threshold.
 - **Impact:** Proves the packet receipt was attested as absent by the quorum.

Branches and code coverage

Intended branches

- Matching path found with zero commitment — returns consensus timestamp.

☒ Test coverage

Negative behavior

- Reverts with `FrozenClientState` if the client is frozen.

- ☐ Negative test
 - Reverts with `InvalidPathLength` if the path length does not equal 1.
- ☒ Negative test
 - Reverts with `ConsensusTimestampNotFound` if there is no timestamp at proof height.
- ☐ Negative test
 - Reverts with signature errors if signature verification fails.
- ☐ Negative test
 - Reverts with `HeightMismatch` if the attested height does not equal proof height.
- ☐ Negative test
 - Reverts with `EmptyPackets` if the packets array is empty.
- ☐ Negative test
 - Reverts with `NotNonMember` if path is found but commitment is nonzero.
- ☒ Negative test
 - Reverts with `NotMember` if path is not found at all.
- ☒ Negative test
 - Rejects cross-domain replay (state signatures rejected as packet signatures).
- ☒ Negative test

Function call analysis

- `_taggedSigningInput(proof.attestationData, ATTESTATION_TYPE_PACKET)`
 - **What is controllable?** `attestationData`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Same domain separation as `verifyMembership` (tag `0x02`).
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `_verifySignaturesThreshold(digest, proof.signatures)`
 - **What is controllable?** `signatures`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — reverts on failure.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Same $O(n^2)$ pattern. The nonmembership proof relies on attestors correctly reporting zero commitment for absent paths — security depends on attesor honesty.

5.2. Module: ICS20Transfer.sol

Function: initialize(address ics26Router, address escrowLogic, address ibcERC20Logic, address permit2, address authority)

This function initializes the proxy from a clean (version 0) state. It deploys new UpgradeableBeacon contracts for IBCERC20 and Escrow logic and stores references to the ICS26Router, permit2, and AccessManager.

Inputs

- ics26Router
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** Stored as the authoritative ICS26 router — all onlyRouter checks compare against this value.
- escrowLogic, ibcERC20Logic
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** Implementation contracts used by all future Escrow and IBCERC20 beacon proxies.
- permit2
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** Used in sendTransferWithPermit2 for signature-based transfers.
- authority
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** Controls all restricted functions (pause, upgrade, role-gated sends).

Branches and code coverage

Intended branches

- Deploys beacons and stores all addresses.
 - ☒ Test coverage
- Initializes Multicall, Pausable, and AccessManaged modules.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidInitialization` if `_getInitializedVersion() != 0` (via `onlyVersion(0)`).
 - ☐ Negative test
- Reverts with `InvalidInitialization` if called a second time (via `reinitializer(2)`).
 - ☐ Negative test

Function: `initializeV2(address authority)`

This is a migration initializer that upgrades a V1 proxy to V2, adding `AccessManagedUpgradeable`. The caller must be the old ICS26Router admin (checked via the deprecated `IDeprecatedIBCUUPSUpgradeable` interface).

Inputs

- `authority`
 - **Control:** Caller-supplied — verified to be a V1 admin.
 - **Constraints:** N/A.
 - **Impact:** Becomes the new authority for all restricted function access.

Branches and code coverage

Intended branches

- Verifies `isAdmin(msg.sender)` on the old ICS26 router, then calls `__AccessManaged_init(authority)`.
 - ☐ Test coverage

Negative behavior

- Reverts with `ICS20Unauthorized` if `msg.sender` is not the old ICS26Router admin.
 - ☐ Negative test
- Reverts with `InvalidInitialization` if `_getInitializedVersion() != 1` (via `onlyVersion(1)`).
 - ☐ Negative test

Function: `onAcknowledgementPacket(IIBCAppCallbacks.OnAcknowledgementPacketC, calldata msg_)`

This is an IBC application callback invoked by the ICS26Router after a sent packet is acknowledged. On error acknowledgment, it refunds tokens to the original sender. On success acknowledgment, it calls the sender's `onAckPacket` callback if implemented.

Inputs

- `msg_`
 - **Control:** ICS26Router-controlled (only the router can call).
 - **Constraints:** `msg_.payload.value` must be ABI-decodable as `FungibleTokenPacketData`; `msg_.acknowledgement` determines the execution path — if it matches `KECCAK256_UNIVERSAL_ERROR_ACK`, the refund path is taken, otherwise the success path is taken; `packetData.sender` must be a valid hex-encoded address in both paths.
 - **Impact:** On error, it triggers token refund; on success, it triggers the optional sender callback.

Branches and code coverage

Intended branches

- Success acknowledgment — the sender implements `IIBCSenderCallbacks`, and the `onAckPacket(true, ...)` callback is triggered.
 - ☒ Test coverage
- Success acknowledgment — the sender does not implement `IIBCSenderCallbacks`, and there is no callback.
 - ☐ Test coverage
- Success acknowledgment — the sender does not implement `IIBCSenderCallbacks`, and the function returns silently.
 - ☐ Test coverage
- Error acknowledgment — IBCERC20 returning to source (was burned on send), and tokens are reminted to escrow then released to sender via `escrow.send`.
 - ☒ Test coverage
- Error acknowledgment — IBCERC20 returning to source — the `onAckPacket(false, ...)` callback is triggered if implemented.
 - ☐ Test coverage
- Error acknowledgment — native ERC-20 (or middle-chain IBCERC20) — tokens are released from escrow to the sender via `escrow.send`.

☒ Test coverage

- Error acknowledgment — native ERC-20 (or middle-chain IBCERC20) — the `onAckPacket(false, ...)` callback is triggered if implemented.

☒ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

- Reverts with `ICS20Unauthorized` if `msg.sender != ics26Router`.

☐ Negative test

- Reverts with `ICS20EscrowNotFound` if no escrow exists for `msg_.sourceClient` (refund path).

☐ Negative test

- Reverts with `ICS20DenomNotFound` if the denom cannot be resolved during refund.

☒ Negative test

- Reverts with `ICS20InvalidAddress` if `packetData.sender` is not a valid hex-encoded address.

☒ Negative test

- Reverts with `RateLimitExceeded` if the escrow's daily outflow limit for the token has been reached when `escrow.send()` is called (error acknowledgment refund path).

☐ Negative test

- Reverts when the contract is paused (`EnforcedPause`).

☐ Negative test

Function: `onRecvPacket(IIBCAppCallbacks.OnRecvPacketCallback calldata msg_)`

This is an IBC application callback invoked by the `ICS26Router` when a cross-chain packet arrives. It decodes the `FungibleTokenPacketData`, determines whether the receiving chain is the origin source of the token (i.e., the denom is prefixed with `sourcePort / sourceClient`), and either releases escrowed tokens or mints new IBCERC20 vouchers to the receiver. It returns an ABI-encoded acknowledgment.

Inputs

- `msg_`
- **Control:** `ICS26Router`-controlled (only the router can call).

- **Constraints:** `msg_.payload.version` must equal `ICS20_VERSION`; `sourcePort` and `destPort` must equal `transfer`; `encoding` must equal `application/x-solidity-abi`; `packetData.amount > 0`; and `packetData.receiver` must be a valid hex-encoded Ethereum address.
- **Impact:** Determines which token is released or minted and to whom.

Branches and code coverage

Intended branches

- Returning-to-origin path (native ERC-20) — escrow releases the native token to the receiver.
☒ Test coverage
- Returning-to-origin path (forwarded IBCERC20) — escrow releases an IBCERC20 token to the receiver.
☒ Test coverage
- First arrival path — new IBCERC20 beacon proxy minted into existence.
☒ Test coverage
- First arrival path — tokens minted to escrow then sent to receiver.
☒ Test coverage
- Subsequent arrival path — existing IBCERC20 tokens minted to escrow then sent to receiver.
☒ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
☐ Negative test
- Reverts with `ICS20Unauthorized` if `msg.sender != ics26Router`.
☐ Negative test
- Reverts with `ICS20UnexpectedVersion` if the payload version is incorrect.
☒ Negative test
- Reverts with `ICS20InvalidPort` if `sourcePort` or `destPort` is not `transfer`.
☒ Negative test
- Reverts with `ICS20UnexpectedEncoding` if the encoding is incorrect.
☒ Negative test
- Reverts with `ICS20InvalidAmount` if `packetData.amount == 0`.
☒ Negative test

- Reverts with `ICS20InvalidAddress` if `packetData.receiver` is not a valid hex-encoded Ethereum address.
 - ☒ Negative test
- Reverts with `ICS20DenomNotFound` if a forwarded `IBCERC20` denom is not registered.
 - ☒ Negative test
- Reverts with `RateLimitExceeded` if the escrow's daily outflow limit for the token has been reached when `escrow.send()` is called.
 - ☒ Negative test
- Reverts when the contract is paused (`EnforcedPause`).
 - ☐ Negative test

Function: `onTimeoutPacket(IIBCAppCallbacks.OnTimeoutPacketCallback calldata msg_)`

This is an IBC application callback invoked by the `ICS26Router` when a sent packet times out. It refunds tokens to the original sender and triggers the sender's `onTimeoutPacket` callback if implemented.

Inputs

- `msg_`
 - **Control:** `ICS26Router`-controlled (only the router can call).
 - **Constraints:** Payload must be decodable as `FungibleTokenPacketData`; `packetData.sender` must be a valid hex address.
 - **Impact:** Triggers token refund and optional sender callback.

Branches and code coverage

Intended branches

- `IBCERC20` returning to source (was burned on send) — tokens reminted to escrow then released to sender via `escrow.send`.
 - ☒ Test coverage
- `IBCERC20` returning to source — sender's `onTimeoutPacket` callback is triggered if implemented.
 - ☐ Test coverage
- Native ERC-20 (or middle-chain `IBCERC20`) — tokens are released from escrow to sender via `escrow.send`.

☒ Test coverage

- Native ERC-20 (or middle-chain IBCERC20) — sender's `onTimeoutPacket` callback is triggered if implemented.

☒ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

- Reverts with `ICS20Unauthorized` if `msg.sender != ics26Router`.

☐ Negative test

- Reverts with `ICS20InvalidAddress` if `packetData.sender` is not a valid hex-encoded address.

☒ Negative test

- Reverts with `ICS20EscrowNotFound` if no escrow exists for the source client.

☐ Negative test

- Reverts with `ICS20DenomNotFound` if the `denom` cannot be resolved.

☒ Negative test

- Reverts with `RateLimitExceeded` if the escrow's daily outflow limit for the token has been reached when `escrow.send()` is called.

☐ Negative test

- Reverts when the contract is paused (`EnforcedPause`).

☐ Negative test

Function: `pause()`

This function pauses all functions guarded by `whenNotPaused` (`sendTransfer`, `sendTransferWithPermit2`, `sendTransferWithSender`, `onRecvPacket`, `onAcknowledgementPacket`, `onTimeoutPacket`).

Branches and code coverage

Intended branches

- The caller has the appropriate `AccessManager` role.

☒ Test coverage

- `_pause()` is called.

☒ Test coverage

- The Paused event is emitted.

☐ Test coverage

Negative behavior

- Reverts with AccessManagedUnauthorized if the caller lacks the required AccessManager role.

☒ Negative test

- Reverts with EnforcedPause if the contract is already paused.

☐ Negative test

Function: `sendTransfer(IICS20TransferMsgs.SendTransferMsg calldata msg_)`

This function initiates a cross-chain token transfer. It pulls tokens from the caller into the per-client Escrow contract (creating the escrow on first use), calls `escrow.recvCallback`, constructs an ICS20 FungibleTokenPacketData, and sends it as an IBC packet via ICS26Router. If the denom is an IBCERC20 returning to its origin chain, the tokens are burned instead of held in escrow.

Inputs

- `msg_.sourceClient`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Routes the packet to the correct counterparty client.
- `msg_.denom`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Determines the token transferred — if it maps to an IBCERC20 denom prefixed by the source client, tokens are burned rather than escrowed.
- `msg_.amount`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be > 0 .
 - **Impact:** Amount transferred and encoded in the packet.
- `msg_.receiver`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Destination address on the counterparty chain.

- `msg_.timeoutTimestamp`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Packet time-out — if exceeded, tokens are refunded.
- `msg_.memo`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Passed verbatim in the packet data.

Branches and code coverage

Intended branches

- Native ERC-20 path — tokens are transferred to escrow, the denom is encoded as hex address, and the packet is sent.
 - ☒ Test coverage
- IBCERC20 returning-to-source path — tokens are transferred to escrow then burned.
 - ☒ Test coverage
- IBCERC20 returning-to-source path — the packet is sent with the full stored IBC denom path (the prefix is not stripped on the send side).
 - ☒ Test coverage
- First transfer for a sourceClient — new Escrow beacon proxy is deployed.
 - ☒ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test
- Reverts with `ICS20InvalidAmount` if `msg_.amount == 0`.
 - ☒ Negative test
- Reverts (ERC-20 error) if the caller has insufficient allowance or balance.
 - ☒ Negative test
- Reverts with `ICS20UnexpectedERC20Balance` if the ERC-20 delivers fewer tokens than expected (fee-on-transfer tokens).
 - ☒ Negative test
- Reverts when the contract is paused (`EnforcedPause`).
 - ☒ Negative test
- After a successful `sendTransfer`, if a rate limit is configured for the token on the escrow,

recvCallback must correctly reduce the daily usage counter by msg_.amount. If the current usage is less than msg_.amount, the counter must clamp to 0 (no underflow). If no rate limit is configured (rateLimit == 0), the counter must remain untouched.

☑ Negative test

Function: `sendTransferWithPermit2(IICS20TransferMsgs.SendTransferMsg calldata msg_, ISignatureTransfer.PermittedTransferFrom calldata permit, bytes calldata signature)`

This function is the same as `sendTransfer` but uses Permit2 for token approval, allowing gasless ERC-20 approvals via an off-chain signature.

Inputs

- msg_
 - **Control:** Caller-controlled.
 - **Constraints:** Same as `sendTransfer` — additionally, `permit.permitted.token` must equal `msg_.denom`.
 - **Impact:** Same as `sendTransfer`.
- permit
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Authorizes token transfer through Permit2.
- signature
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Authorizes the Permit2 transfer on behalf of the token owner.

Branches and code coverage

Intended branches

- Native ERC-20 path — tokens are transferred via Permit2 to escrow, the denom is encoded as a hex address, and the packet is sent.
 - ☑ Test coverage
- IBCERC20 returning-to-source path — tokens are transferred via Permit2 to escrow then burned.
 - ☑ Test coverage
- IBCERC20 returning-to-source path — the packet is sent with the full stored IBC denom

path (prefix is not stripped on the send side).

☐ Test coverage

- First transfer for a sourceClient — new Escrow beacon proxy is deployed.

☒ Test coverage

Negative behavior

- Reverts with ReentrancyGuardReentrantCall if called reentrantly.

☐ Negative test

- Reverts with ICS20InvalidAmount if msg_.amount == 0.

☒ Negative test

- Reverts with ICS20Permit2TokenMismatch if permit.permitted.token != msg_.denom.

☒ Negative test

- Reverts (Permit2 error) if the signature is invalid or expired.

☒ Negative test

- Reverts with ICS20UnexpectedERC20Balance for fee-on-transfer tokens.

☒ Negative test

- Reverts when the contract is paused (EnforcedPause).

☐ Negative test

- After a successful sendTransferWithPermit2, if a rate limit is configured for the token on the escrow, recvCallback must correctly reduce the daily usage counter by msg_.amount. If the current usage is less than msg_.amount, the counter must clamp to 0 (no underflow). If no rate limit is configured (rateLimit == 0), the counter must remain untouched.

☒ Negative test

Function: `sendTransferWithSender(IICS20TransferMsgs.SendTransferMsg calldata msg_, address sender)`

This function is a restricted variant of [sendTransfer](#) that allows a privileged caller to specify an arbitrary sender address. Tokens are still pulled from msg_.sender (the caller), but the sender field in the packet data (used for refunds) is set to the provided sender argument. It is intended for contracts that hold tokens on behalf of users.

Inputs

- msg_

- **Control:** Restricted and caller-controlled.
 - **Constraints:** Same as `sendTransfer`.
 - **Impact:** Same as `sendTransfer`.
- sender
 - **Control:** Restricted and caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Used as the sender field in the ICS20 packet — also the address that receives refunded tokens on time-out or error acknowledgment.

Branches and code coverage

Intended branches

- Native ERC-20 path — tokens are pulled from `msg.sender` to escrow.
 - ☒ Test coverage
- Native ERC-20 path — the denom is encoded as hex address.
 - ☒ Test coverage
- Native ERC-20 path — the packet is sent with a custom sender.
 - ☒ Test coverage
- IBCERC20 returning-to-source path — tokens are transferred to escrow then burned.
 - ☐ Test coverage
- IBCERC20 returning-to-source path — the packet is sent with the full stored IBC denom path (prefix is not stripped on the send side) and custom sender.
 - ☐ Test coverage
- First transfer for a `sourceClient` — new Escrow beacon proxy is deployed.
 - ☐ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test
- Reverts with `AccessManagedUnauthorized` if the caller is not authorized by the `AccessManager`.
 - ☐ Negative test
- Reverts with `ICS20InvalidAmount` if `msg.amount == 0`.
 - ☐ Negative test
- Reverts when the contract is paused (`EnforcedPause`).
 - ☐ Negative test

- After a successful `sendTransferWithSender`, if a rate limit is configured for the token on the escrow, `recvCallback` must correctly reduce the daily usage counter by `msg_.amount`. If the current usage is less than `msg_.amount`, the counter must clamp to 0 (no underflow). If no rate limit is configured (`rateLimit == 0`), the counter must remain untouched.

☒ Negative test

Function: `setCustomERC20(string calldata denom, address token)`

This function registers a custom ERC20 contract for a given IBC denom. It must be called before the first `onRecvPacket` for that denom, since `onRecvPacket` autocreates an IBCERC20 beacon proxy via `_getOrCreateIBCERC20` and registers it in `_ibcERC20Contracts`, which would cause this function to revert. It requires authorization via the `AccessManagedUpgradeable` (restricted modifier).

Inputs

- `denom`
 - **Control:** Restricted, caller-controlled.
 - **Constraints:** Must not already be registered in `_ibcERC20Contracts`.
 - **Impact:** The denom key used to look up the custom ERC-20 in all future transfers.
- `token`
 - **Control:** Restricted, caller-controlled.
 - **Constraints:** Must not already be registered in `_ibcERC20Denoms`.
 - **Impact:** The ERC-20 that will be minted/burned when this denom is received or sent.

Branches and code coverage

Intended branches

- Registers the denom-to-token bidirectional mapping.

☒ Test coverage

Negative behavior

- Reverts with `AccessManagedUnauthorized` if the caller lacks the required role.

☐ Negative test

- Reverts with `ICS20DenomAlreadyExists` if the denom is already registered.

☐ Negative test

- Reverts with `ICS20TokenAlreadyExists` if the token address is already registered under a different denom.

☐ Negative test

Function: `unpause()`

This function unpauses the contract, reenabling all `whenNotPaused` functions.

Branches and code coverage

Intended branches

- The caller has the appropriate `AccessManager` role.

☒ Test coverage

- `_unpause()` is called.

☒ Test coverage

- The `Unpaused` event is emitted.

☐ Test coverage

Negative behavior

- Reverts with `AccessManagedUnauthorized` if the caller lacks the required `AccessManager` role.

☒ Negative test

- Reverts with `ExpectedPause` if the contract is not currently paused.

☐ Negative test

Function: `upgradeEscrowTo(address newEscrowLogic)`

This function upgrades the implementation of the Escrow `UpgradeableBeacon`, affecting all existing and future Escrow beacon proxies.

Inputs

- `newEscrowLogic`
 - **Control:** Restricted and caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Immediately changes the logic used by all Escrow contracts across all clients.

Branches and code coverage

Intended branches

- The caller has the required AccessManager role.
 - ☒ Test coverage
- Beacon upgradeTo succeeds.
 - ☒ Test coverage

Negative behavior

- Reverts with AccessManagedUnauthorized if the caller lacks the required role.
 - ☒ Negative test
- Reverts (OZ beacon error) if newEscrowLogic is not a contract.
 - ☐ Negative test

Function: upgradeIBERC20To(address newIBERC20Logic)

This function upgrades the implementation of the IBCERC20 UpgradeableBeacon, affecting all existing and future IBCERC20 beacon proxies.

Inputs

- newIBERC20Logic
 - **Control:** Restricted and caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Immediately changes the logic used by all IBCERC20 token contracts.

Branches and code coverage

Intended branches

- The caller has the required AccessManager role.
 - ☒ Test coverage
- Beacon upgradeTo succeeds.
 - ☒ Test coverage

Negative behavior

- Reverts with AccessManagedUnauthorized if the caller lacks the required role.

- ☒ Negative test
- Reverts (OZ beacon error) if newIBCEC20Logic is not a contract.
- ☐ Negative test

5.3. Module: ICS26Router.sol

Function: `ackPacket(IICS26RouterMsgs.MsgAckPacket calldata msg_)`

This function processes an acknowledgment for a previously sent packet on the source chain. It verifies the acknowledgment commitment proof against the light client, deletes the packet commitment, and dispatches the acknowledgment callback to the source IBC app. It is access-controlled via `restricted`.

Inputs

- `msg_.packet`
 - **Control:** Relay-controlled (but the commitment is verified against stored state).
 - **Constraints:** Must have exactly one payload; `sourceClient` counterparty must match `destClient`.
 - **Impact:** Identifies the packet being acknowledged and the app to notify.
- `msg_.acknowledgement`
 - **Control:** Relay-controlled (verified by light-client proof against destination chain).
 - **Constraints:** Must match the acknowledgment committed on the destination chain (verified via proof).
 - **Impact:** Passed to the IBC app's `onAcknowledgementPacket` callback.
- `msg_.proofAked`
 - **Control:** Relay-controlled.
 - **Constraints:** Must pass light-client `verifyMembership`.
 - **Impact:** Proves the acknowledgment was committed on the destination chain.
- `msg_.proofHeight`
 - **Control:** Relay-controlled.
 - **Constraints:** Must correspond to a trusted height.
 - **Impact:** Height at which the proof is valid.

Branches and code coverage

Intended branches

- Proof is verified, commitment is deleted, and the app callback is executed.
 - ☒ Test coverage
- Packet commitment is not found — emits Noop and returns early.
 - ☐ Test coverage

Negative behavior

- Reverts with `IBCMultiPayloadPacketNotSupported` if there are multiple payloads.
 - ☐ Negative test
- Reverts with `IBCInvalidCounterparty` if there is a counterparty mismatch.
 - ☐ Negative test
- Reverts if proof verification fails (light-client revert).
 - ☐ Negative test
- Reverts with `IBCPacketCommitmentMismatch` if the packet commitment exists but does not match.
 - ☐ Negative test
- Reverts if app `onAcknowledgementPacket` reverts (no try-catch unlike `recvPacket`).
 - ☐ Negative test
- Reverts with `AccessManagedUnauthorized` if the caller lacks the relayer role.
 - ☐ Negative test
- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test

Function call analysis

- `getClient(msg_.packet.sourceClient).verifyMembership(membershipMsg)`
 - **What is controllable?** `proofAcked` and `proofHeight`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Return value unused. Reverts on failure.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert propagates.
- `checkAndDeletePacketCommitment(msg_.packet)`
 - **What is controllable?** `packet`.
 - **If the return value is controllable, how is it used and how can it go wrong?**

Returns `false` if no commitment exists (no-op). Returns `true` if found and deleted.

- **What happens if it reverts, reenters or does other unusual control flow?**

Reverts with `IBCPacketCommitmentMismatch` if the stored commitment does not match. This prevents acknowledging a different packet than what was sent.

- `getIBCApp(payload.sourcePort).onAcknowledgementPacket(...)`

- **What is controllable?** acknowledgement.

- **If the return value is controllable, how is it used and how can it go wrong?**

N/A.

- **What happens if it reverts, reenters or does other unusual control flow?**

N/A.

Function: `addIBCApp(address app)`

This is a permissionless variant that registers an IBC application contract using the app's hex address as its port identifier. Any address can call this function to register any contract as an IBC app.

Inputs

- `app`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A (no zero-address or interface check).
 - **Impact:** The app address becomes the registered IBC application for its own hex-string port. Once registered, the app receives IBC callbacks for packets addressed to its port.

Branches and code coverage

Intended branches

- The port ID is generated from the app's hex address, and the app is registered.

☒ Test coverage

Negative behavior

- Reverts with `IBCPortAlreadyExists` if the port is already registered.

☐ Negative test

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

Function call analysis

- `Strings.toHexString(app)`
 - **What is controllable?** `app`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Produces a deterministic hex string port ID. No collision with custom port IDs since `addIBCApp(string, address)` rejects address-formatted strings.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `_addIBCApp(portId, app)`
 - **What is controllable?** `portId` and `app`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `addIBCApp(string calldata portId, address app)`

This function is an access-controlled variant that registers an IBC application under a custom (nonaddress) port identifier. It is only callable by addresses with the appropriate role via the restricted modifier (`AccessManaged`).

Inputs

- `portId`
 - **Control:** Admin-controlled (requires restricted role).
 - **Constraints:** Must be nonempty, must not parse as a valid Ethereum address, and must pass `IBCIdentifiers.validateCustomIBCIdentifier`.
 - **Impact:** The custom port ID under which the app is registered. Validation prevents collision with permissionless address-based port IDs.
- `app`
 - **Control:** Admin-controlled.
 - **Constraints:** N/A (no zero-address or interface check).
 - **Impact:** The app address receives IBC callbacks for the given port.

Branches and code coverage

Intended branches

- The port ID passes all validations, and the app is registered.

☒ Test coverage

Negative behavior

- Reverts with `IBCInvalidPortIdentifier` if `portId` is empty.

☐ Negative test

- Reverts with `IBCInvalidPortIdentifier` if `portId` is a valid address.

☒ Negative test

- Reverts with `IBCInvalidPortIdentifier` if `portId` fails custom IBC identifier validation.

☐ Negative test

- Reverts with `IBCPortAlreadyExists` if the port already exists.

☒ Negative test

- Reverts with `AccessManagedUnauthorized` if the caller lacks access.

☒ Negative test

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

Function call analysis

- `Strings.tryParseAddress(portId)`
 - **What is controllable?** `portId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns `(true, addr)` if the string is a valid address, causing a revert.
Ensures no collision between custom ports and permissionless address ports.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `IBCIdentifiers.validateCustomIBCIdentifier(bytes(portId))`
 - **What is controllable?** `portId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns `true` if the identifier is valid. If `false`, the function reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `recvPacket(IICS26RouterMsgs.MsgRecvPacket calldata msg_)`

This function receives and processes an IBC packet on the destination chain.

It verifies the packet commitment proof against the light client, sets the packet receipt to prevent

replay, dispatches the payload to the destination IBC app, and commits the acknowledgment. It is access-controlled via restricted.

Inputs

- `msg_.packet`
 - **Control:** Relay-controlled (but content verified by light-client proof).
 - **Constraints:** Must have exactly one payload; `destClient` counterparty must match `sourceClient`; and `timeoutTimestamp` must be greater than `block.timestamp`.
 - **Impact:** Determines which app processes the packet, which client verifies the proof, and the content delivered to the app.
- `msg_.proofCommitment`
 - **Control:** Relay-controlled.
 - **Constraints:** Must pass light-client `verifyMembership`.
 - **Impact:** Cryptographic proof that the packet commitment exists on the source chain.
- `msg_.proofHeight`
 - **Control:** Relay-controlled.
 - **Constraints:** Must correspond to a trusted height in the light client.
 - **Impact:** The height at which the proof is valid.

Branches and code coverage

Intended branches

- The proof is verified, receipt is set, app callback succeeds, and acknowledgment is committed.
 - ☒ Test coverage
- Packet receipt already set — emits Noop and returns early.
 - ☐ Test coverage
- The app `onRecvPacket` reverts with nonempty reason — universal error acknowledgment is committed.
 - ☒ Test coverage

Negative behavior

- Reverts with `IBCMultiPayloadPacketNotSupported` if there are multiple payloads.
 - ☐ Negative test
- Reverts with `IBCInvalidCounterparty` if there is a counterparty mismatch.

- ☐ Negative test
 - Reverts with `IBCInvalidTimeoutTimestamp` if the time-out is expired.
- ☐ Negative test
 - Reverts if proof verification fails (light-client revert).
- ☒ Negative test
 - Reverts with `IBCAsyncAcknowledgementNotSupported` if the app returns an empty acknowledgment.
- ☐ Negative test
 - Reverts with `IBCErrorUniversalAcknowledgement` if app returns a universal error acknowledgment directly.
- ☐ Negative test
 - Reverts with `IBCFailedCallback` if the app reverts with an empty reason (covers being out of gas).
- ☒ Negative test
 - Reverts with `AccessManagedUnauthorized` if the caller lacks the relayer role.
- ☐ Negative test
 - Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
- ☐ Negative test

Function call analysis

- `getClient(msg_.packet.destClient).verifyMembership(membershipMsg)`
 - **What is controllable?** `proofCommitment` and `proofHeight`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Return value is unused (slither comment confirms). The function reverts on failure.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `setPacketReceipt(msg_.packet)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns `false` if receipt already exists (triggers no-op). Returns `true` if newly set.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `getIBCApp(payload.destPort).onRecvPacket(...)`
 - **What is controllable?** `payload`.

- **If the return value is controllable, how is it used and how can it go wrong?**
Must be nonempty and must not be the universal error acknowledgment. The acknowledgment is committed on chain. If the app returns a malicious acknowledgment, it is committed as is.
- **What happens if it reverts, reenters or does other unusual control flow?**
Caught by try-catch. For a nonempty revert reason, a universal error acknowledgment is committed. For an empty revert reason (out of gas), the entire transaction reverts with IBCFailedCallback.
- `commitPacketAcknowledgement(msg_.packet, acks)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `sendPacket(IICS26RouterMsgs.MsgSendPacket calldata msg_)`

This function sends an IBC packet from a registered IBC application. The caller must be the registered app for the source port. It validates time-out constraints, increments the sequence counter, commits the packet, and emits a `SendPacket` event.

Inputs

- `msg_.payload.sourcePort`
 - **Control:** IBC-app-controlled (must match the calling app's registered port).
 - **Constraints:** Must map to a registered IBC app whose address equals `_msgSender()`.
 - **Impact:** Core authorization check — only the registered app can send on its own port.
- `msg_.sourceClient`
 - **Control:** IBC-app-controlled.
 - **Constraints:** Must have a registered counterparty.
 - **Impact:** Determines the IBC client and counterparty chain.
- `msg_.timeoutTimestamp`
 - **Control:** IBC-app-controlled (ultimately user-controlled via the app).
 - **Constraints:** Must be greater than `block.timestamp`; must be within `MAX_TIMEOUT_DURATION` (one day) of current time.
 - **Impact:** Controls packet expiry.
- `msg_.payload(destPort, version, encoding, value)`
 - **Control:** IBC-app-controlled.

- **Constraints:** N/A at router level.
- **Impact:** Included in the committed packet. Validated by the destination app.

Branches and code coverage

Intended branches

- The caller matches the registered app, the packet is committed, and the sequence is returned.

☒ Test coverage

Negative behavior

- Reverts with `IBCApplNot Found` if the source port has no registered app.

☐ Negative test

- Reverts with `IBCUnauthorizedSender` if the caller is not the registered app.

☒ Negative test

- Reverts if there is no counterparty for the source client.

☐ Negative test

- Reverts with `IBCInvalidTimeoutTimestamp` if time-out is in the past.

☐ Negative test

- Reverts with `IBCInvalidTimeoutDuration` if time-out duration exceeds one day.

☒ Negative test

- Reverts with `IBCPacketCommitmentAlreadyExists` if the commitment already exists.

☐ Negative test

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

Function call analysis

- `getIBCAppl(msg_.payload.sourcePort)`

- **What is controllable?** `msg_.payload.sourcePort`.

- **If the return value is controllable, how is it used and how can it go wrong?**
Returns the registered app address, compared against `_msgSender()`. Not controllable.

- **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

- `getCounterparty(msg_.sourceClient)`

- **What is controllable?** `msg_.sourceClient`.
- **If the return value is controllable, how is it used and how can it go wrong?**
Returns counterparty information, including remote client ID and Merkle prefix. Used to populate `destClient` in the packet.
- **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `nextSequenceSend(msg_.sourceClient)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the atomically incremented sequence number. Not controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `commitPacket(packet)`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `timeoutPacket(IICS26RouterMsgs.MsgTimeoutPacket calldata msg_)`

This function processes a time-out for a previously sent packet that was not received within the time-out window. It verifies nonmembership of the packet receipt on the destination chain, verifies that the counterparty timestamp meets or exceeds the time-out, deletes the packet commitment, and dispatches the time-out callback. It is access-controlled via `restricted`.

Inputs

- `msg_.packet`
 - **Control:** Relay-controlled (but the commitment is verified against stored state).
 - **Constraints:** Must have exactly one payload; `sourceClient` counterparty must match `destClient`.
 - **Impact:** Identifies the timed-out packet and the app to notify.
- `msg_.proofTimeout`
 - **Control:** Relay-controlled.
 - **Constraints:** Must pass light-client `verifyNonMembership`.
 - **Impact:** Proves the packet receipt does *not* exist on the destination chain.
- `msg_.proofHeight`

- **Control:** Relay-controlled.
- **Constraints:** Must correspond to a trusted height.
- **Impact:** Height at which the nonmembership proof is valid.

Branches and code coverage

Intended branches

- The nonmembership proof is verified, the commitment is deleted, and the time-out callback is dispatched.
 - ☒ Test coverage
- The packet commitment is not found — emits Noop and returns early.
 - ☐ Test coverage

Negative behavior

- Reverts with `IBCMultiPayloadPacketNotSupported` if there are multiple payloads.
 - ☐ Negative test
- Reverts with `IBCInvalidCounterparty` if there is a counterparty mismatch.
 - ☐ Negative test
- Reverts if the nonmembership proof fails (light-client revert).
 - ☐ Negative test
- Reverts with `IBCInvalidTimeoutTimestamp` if the counterparty timestamp is less than the packet time-out.
 - ☐ Negative test
- Reverts with `IBCPacketCommitmentMismatch` if the commitment exists but does not match.
 - ☐ Negative test
- Reverts if `app.onTimeoutPacket` reverts (no try-catch).
 - ☐ Negative test
- Reverts with `AccessManagedUnauthorized` if the caller lacks the relay role.
 - ☐ Negative test
- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test

Function call analysis

- `getClient(msg_.packet.sourceClient).verifyNonMembership(nonMembershipMsg)`

- **What is controllable?** proofTimeout and proofHeight.
- **If the return value is controllable, how is it used and how can it go wrong?**
Returns the counterparty timestamp at the proof height. Compared against packet.timeoutTimestamp — must be greater than or equal to time-out.
- **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- checkAndDeletePacketCommitment(msg_.packet)
 - **What is controllable?** packet.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns false if not found (no-op). Returns true if found and deleted.
Prevents double time-out or acknowledgment after time-out.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- getIBCAppl(payload.sourcePort).onTimeoutPacket(...)
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

5.4. Module: ICS27Account.sol

Function: delegateExecute(address target, bytes calldata data)

This function performs a delegatecall to target using the account's storage and ETH balance context. It is only callable by the account itself.

Inputs

- target
 - **Control:** Indirectly ICS27GMP-controlled (via GMP payload).
 - **Constraints:** N/A.
 - **Impact:** Executes arbitrary code in the context of this account — can read/write all storage slots and forward ETH.
- data
 - **Control:** Indirectly ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** Full arbitrary code execution in the account's context.

Branches and code coverage

Intended branches

- The `delegatecall` succeeds.
 - ☒ Test coverage
- The result is returned.
 - ☒ Test coverage

Negative behavior

- Reverts with `ICS27Unauthorized` if `msg.sender != address(this)`.
 - ☒ Negative test
- Reverts with `AddressEmptyCode` if target has no code.
 - ☐ Negative test
- Reverts if the `delegatecall` reverts.
 - ☒ Negative test

Function: `execute(address target, bytes calldata data, uint256 value)`

This function performs a value-bearing low-level call to target. It is only callable by the account itself. It is intended to be invoked as a nested call from within `functionCall`.

Inputs

- `target`
 - **Control:** Indirectly ICS27GMP-controlled (via GMP payload).
 - **Constraints:** N/A.
 - **Impact:** The contract is called — arbitrary contract interaction with ETH value.
- `data`
 - **Control:** Indirectly ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** Full arbitrary code execution with ETH value at the target.
- `value`
 - **Control:** Indirectly ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** ETH forwarded to the target.

Branches and code coverage

Intended branches

- The target is called with value.
 - ☒ Test coverage
- The result is returned.
 - ☒ Test coverage

Negative behavior

- Reverts with `ICS27Unauthorized` if `msg.sender != address(this)`.
 - ☒ Negative test
- Reverts with `AddressEmptyCode` if target has no code.
 - ☐ Negative test
- Reverts with `Errors.InsufficientBalance` if the account has insufficient ETH for value.
 - ☒ Negative test
- Reverts if the target call reverts.
 - ☒ Negative test

Function: `executeBatch(IICS27AccountMsgs.Call[] calldata calls)`

This function executes a batch of (target, data, value) calls sequentially. It is only callable by the account itself. Each call uses `Address.functionCallWithValue`.

Inputs

- `calls`
 - **Control:** Indirectly ICS27GMP-controlled (via GMP payload).
 - **Constraints:** N/A.
 - **Impact:** Sequential arbitrary contract interactions with optional ETH value — a single call failure reverts the entire batch.

Branches and code coverage

Intended branches

- All calls succeed.
 - ☒ Test coverage

- The results array is returned.
 - ☒ Test coverage
- The calls array is empty.
 - ☐ Test coverage
- Empty results are returned without error.
 - ☐ Test coverage

Negative behavior

- Reverts with ICS27Unauthorized if `msg.sender != address(this)`.
 - ☒ Negative test
- Reverts with AddressEmptyCode if any `call.target` has no code.
 - ☐ Negative test
- Reverts with Errors.InsufficientBalance if the account has insufficient ETH for any `call.value`.
 - ☒ Negative test
- Reverts if any single call in the batch reverts (the entire batch is reverted).
 - ☒ Negative test

Function: `functionCall(address target, bytes calldata data)`

This function performs a low-level `call` to `target` with `data`. It is only callable by the ICS27GMP contract. This is the primary entry point by which ICS27GMP executes cross-chain GMP payloads inside this account. Authorization relies entirely on the IBC protocol; ICS26Router verifies the packet via light client, then ICS27GMP calls this function. This account is deterministically assigned to a unique counterparty (`clientId`, `sender`, `salt`) triplet via CREATE2, so each counterparty address has its own isolated account. There is no on-chain allowlist for `target` — any contract on this chain can be called.

Inputs

- `target`
 - **Control:** ICS27GMP-controlled (driven by the counterparty chain's GMP message).
 - **Constraints:** N/A.
 - **Impact:** The contract that is called. No allowlist — full arbitrary contract interaction on this chain.
- `data`
 - **Control:** ICS27GMP-controlled.

- **Constraints:** N/A.
- **Impact:** The call payload — full arbitrary code execution within the target.

Branches and code coverage

Intended branches

- `msg.sender` is `_ics27`.
☒ Test coverage
- The call succeeds.
☒ Test coverage
- The result is returned.
☒ Test coverage

Negative behavior

- Reverts with `ICS27Unauthorized` if `msg.sender != _ics27`.
☒ Negative test
- Reverts with `AddressEmptyCode` if target has no code.
☐ Negative test
- Reverts if the target call reverts.
☒ Negative test

Function: `initialize(address ics27_)`

This function initializes the proxy and stores the ICS27GMP contract address.

Inputs

- `ics27_`
 - **Control:** Deployer-controlled (the ICS27GMP contract deploys this proxy).
 - **Constraints:** N/A.
 - **Impact:** Stored as the sole authorized caller for `functionCall`.

Branches and code coverage

Intended branches

- Stores `ics27_`.

- ☒ Test coverage
- Calls `__Context_init()`.
- ☐ Test coverage

Negative behavior

- Reverts with `InvalidInitialization` if called on an already initialized proxy.
- ☐ Negative test

5.5. Module: ICS27GMP.sol

Function: `onAcknowledgementPacket(IIBCAppCallbacks.OnAcknowledgementPacketC calldata msg_)`

This function processes an acknowledgment for a previously sent ICS27GMP packet. It decodes the original packet data to recover the sender address and then dispatches a callback to the sender via `IBCSenderCallbacksLib.ackPacketCallback`, indicating success or failure based on whether the acknowledgment matches the universal error acknowledgment.

Inputs

- `msg_`
 - **Control:** ICS26Router-controlled (only the router can call via `onlyRouter`).
 - **Constraints:** `msg_.payload.value` must ABI-decode as `GMPPacketData`; decoded `packetData.sender` must parse as a valid Ethereum address.
 - **Impact:** Determines which address receives the acknowledgment callback and whether it indicates success or failure.

Branches and code coverage

Intended branches

- Sender address is parsed successfully, and success callback is dispatched (acknowledgment does not equal universal error acknowledgment).
 - ☒ Test coverage
- Sender address is parsed successfully, and failure callback is dispatched (acknowledgment equals universal error acknowledgment).
 - ☒ Test coverage

Negative behavior

- Reverts with `ICS27InvalidSender` if the sender string is not a valid address.

- ☒ Negative test
 - Reverts if `msg_.payload.value` cannot be ABI-decoded.
- ☒ Negative test
 - Reverts with `ICS27Unauthorized` if the caller is not the `ICS26Router`.
- ☐ Negative test
 - Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
- ☐ Negative test

Function call analysis

- `abi.decode(msg_.payload.value, (GMPPacketData))`
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The sender field is extracted for the callback target. Not attacker-controllable at acknowledgment time.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `IBCSenderCallbacksLib.ackPacketCallback(sender, success, msg_)`
 - **What is controllable?** sender, success, and msg_.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `onRecvPacket(IIBCAppCallbacks.OnRecvPacketCallback calldata msg_)`

This function processes an incoming IBC packet on the destination chain. It decodes the GMP packet data, creates or retrieves an interchain account (ICA) deterministically based on (`clientId`, `sender`, `salt`), and executes the payload as a function call from the ICA to the specified receiver address.

Inputs

- `msg_`
 - **Control:** `ICS26Router`-controlled (only the router can call via `onlyRouter`).
 - **Constraints:** `msg_.payload.version` must equal `ICS27_VERSION`; `sourcePort` and `destPort` must equal `DEFAULT_PORT_ID`; encoding must equal `ICS27_ENCODING`; decoded `packetData.payload` must be nonempty;

and packetData.receiver must be a valid Ethereum address.

- **Impact:** Determines which ICA executes what calldata on which target contract.

Branches and code coverage

Intended branches

- All four field validations pass (version, sourcePort, encoding, destPort).
☒ Test coverage
- Packet data is decoded, and the payload is nonempty.
☒ Test coverage
- The receiver address parses successfully.
☒ Test coverage
- Account does not exist — created via CREATE2 and stored.
☒ Test coverage
- Account already exists — returned from storage.
☒ Test coverage
- Function call to receiver succeeds — acknowledgment returned.
☒ Test coverage

Negative behavior

- Reverts with ICS27UnexpectedVersion if there is a version mismatch.
☒ Negative test
- Reverts with ICS27InvalidPort if there is a source-port mismatch.
☒ Negative test
- Reverts with ICS27InvalidPort if there is a destination-port mismatch.
☒ Negative test
- Reverts with ICS27UnexpectedEncoding if there is an encoding mismatch.
☒ Negative test
- Reverts with ICS27PayloadEmpty if the decoded payload is empty.
☒ Negative test
- Reverts with ICS27InvalidReceiver if the receiver is not a valid address.
☒ Negative test
- Reverts with ICS27Unauthorized if the caller is not the ICS26Router.

- ☐ Negative test
 - Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
- ☐ Negative test
 - Reverts if `account.functionCall` reverts (target call failure).
- ☒ Negative test

Function call analysis

- `abi.decode(msg_.payload.value, (GMPPacketData))`
 - **What is controllable?** `msg_.payload.value`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Decoded struct fields (sender, receiver, salt, payload, memo) drive the entire function logic.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `_getOrCreateAccount(accountId)`
 - **What is controllable?** `accountId`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the ICA address. Each unique (`clientId`, sender, salt) gets a unique account via `CREATE2`. An attacker on the source chain can create arbitrary accounts by varying the salt.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- `account.functionCall(receiver, packetData.payload)`
 - **What is controllable?** receiver and payload.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Return data is wrapped in `ICS27Lib.acknowledgement()` and returned as the acknowledgment. Not security-critical.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

Function: `onTimeoutPacket(IIBCAppCallbacks.OnTimeoutPacketCallback calldata msg_)`

This function processes a time-out for a previously sent ICS27GMP packet that was not received within the time-out window. It decodes the original packet data to recover the sender address and dispatches a time-out callback to the sender via `IBCSenderCallbacksLib.timeoutPacketCallback`.

Inputs

- msg_
 - **Control:** ICS26Router-controlled (only the router can call via onlyRouter).
 - **Constraints:** msg_.payload.value must ABI-decode as GMPPacketData; decoded packetData.sender must parse as a valid Ethereum address.
 - **Impact:** Determines which address receives the time-out callback.

Branches and code coverage

Intended branches

- The sender address is parsed successfully, and the time-out callback is dispatched.

☒ Test coverage

Negative behavior

- Reverts with ICS27InvalidSender if the sender string is not a valid address.

☒ Negative test

- Reverts if msg_.payload.value cannot be ABI-decoded.

☒ Negative test

- Reverts with ICS27Unauthorized if the caller is not the ICS26Router.

☐ Negative test

- Reverts with ReentrancyGuardReentrantCall if called reentrantly.

☐ Negative test

Function call analysis

- abi.decode(msg_.payload.value, (GMPPacketData))
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The sender field is extracted for the callback. Not attacker-controllable at time-out time.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- IBCSenderCallbacksLib.timeoutPacketCallback(sender, msg_)
 - **What is controllable?** sender and msg_.
 - **If the return value is controllable, how is it used and how can it go wrong?**
N/A — no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**

N/A.

Function: `sendCall(IICS27GMPMsgs.SendCallMsg calldata msg_)`

This function allows any external caller to send a cross-chain GMP packet via ICS27. The caller's address is embedded as the sender (checksum hex string) in the packet data. It constructs a `GMPPacketData` struct and forwards it to `ICS26Router.sendPacket`.

Inputs

- `msg_.sourceClient`
 - **Control:** Caller-controlled.
 - **Constraints:** Validated downstream by `ICS26Router.sendPacket` (counterparty must exist).
 - **Impact:** Determines which IBC client the packet is routed through.
- `msg_.timeoutTimestamp`
 - **Control:** Caller-controlled.
 - **Constraints:** Validated by `ICS26Router.sendPacket` (must be greater than `block.timestamp` and within one day).
 - **Impact:** Controls packet expiry window.
- `msg_.receiver`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A at send time. Validated on the receiving side via `Strings.tryParseAddress`.
 - **Impact:** Arbitrary string that becomes the target address on the destination chain.
- `msg_.salt`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.
 - **Impact:** Combined with sender and `clientId` to determine the interchain account identity on the destination.
- `msg_.payload`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty.
 - **Impact:** The raw calldata executed by the interchain account on the destination.
- `msg_.memo`
 - **Control:** Caller-controlled.
 - **Constraints:** N/A.

- **Impact:** Informational field included in the packet data.

Branches and code coverage

Intended branches

- The payload is nonempty — packet construction proceeds.
☒ Test coverage
- The packet is sent via ICS26Router, and the sequence number is returned.
☒ Test coverage

Negative behavior

- Reverts with ICS27PayloadEmpty if msg_.payload is empty.
☒ Negative test
- Reverts with ReentrancyGuardReentrantCall if called reentrantly.
☐ Negative test
- Reverts if ICS26Router.sendPacket reverts (invalid client, time-out, unauthorized port).
☒ Negative test

Function call analysis

- Strings.toChecksumHexString(_msgSender())
 - **What is controllable?** N/A.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Produces the sender string in the packet. Not controllable — cannot be spoofed.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.
- _getICS27GMPStorage()._ics26.sendPacket(MsgSendPacket)
 - **What is controllable?** sourceClient, timeoutTimestamp, and value.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns a uint64 sequence number, passed back to the caller. Not controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?**
N/A.

5.6. Module: IFTAccessManaged.sol

Function: `initialize(address authority_, string calldata ERC20Name, string calldata ERC20Symbol, address ICS27Gmp)`

This function initializes the UUPS proxy, sets the AccessManager authority, and initializes the ERC-20 token and IFT base state.

Inputs

- `authority_`
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** The AccessManager that controls authority-gated functions.
- `ERC20Name`
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** The ERC-20 token name — immutable after initialization (standard ERC-20 convention).
- `ERC20Symbol`
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** The ERC-20 token symbol — immutable after initialization.
- `ICS27Gmp`
 - **Control:** Deployer-controlled.
 - **Constraints:** N/A.
 - **Impact:** Stored as the ICS27GMP contract address — used for sending GMP calls and authorizing callbacks.

Branches and code coverage

Intended branches

- All modules are initialized.
 - ☒ Test coverage
- Storage is set correctly.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidInitialization` if called on an already initialized proxy.
- ☐ Negative test

5.7. Module: IFTBaseUpgradeable.sol

Function: `iftMint(address receiver, uint256 amount)`

This function is called by an `ICS27Account` (deployed by `ICS27GMP`) to mint tokens on this chain when a cross-chain IFT transfer arrives. It validates that the calling account corresponds to the registered counterparty IFT address for the appropriate client and then mints `amount` tokens to `receiver`.

Inputs

- `receiver`
 - **Control:** `ICS27Account`-controlled (ultimately driven by the counterparty chain's GMP call).
 - **Constraints:** N/A.
 - **Impact:** Address that receives the newly minted tokens.
- `amount`
 - **Control:** `ICS27Account`-controlled.
 - **Constraints:** N/A.
 - **Impact:** Amount of tokens minted.

Branches and code coverage

Intended branches

- `msg.sender` is an `ICS27Account` whose `accountId` matches the registered bridge.
- ☒ Test coverage
- Tokens are minted to `receiver`.
- ☒ Test coverage
- `IFTMintReceived` is emitted.
- ☒ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
- ☐ Negative test

- Reverts with `IFTBridgeNotFound` if no bridge is registered for the account's `clientId`.
 - ☑ Negative test
- Reverts with `IFTUnauthorizedMint` if the account's sender field does not match `bridge.counterpartyIFTAddress`.
 - ☑ Negative test
- Reverts with `IFTUnexpectedSalt` if the account was deployed with a nonempty salt.
 - ☑ Negative test
- Reverts with `ICS27AccountNotFound` if `msg.sender` is not a recognized `ICS27Account` (i.e., `getAccountIdentifier` reverts).
 - ☑ Negative test

Function: `iftTransfer(string calldata clientId, string calldata receiver, uint256 amount, uint64 timeoutTimestamp)`

This function initiates a cross-chain IFT transfer with an explicit time-out. It burns `amount` tokens from the caller, sends a GMP call to the counterparty IFT contract via `ICS27GMP`, and records a pending transfer for callback handling.

Inputs

- `clientId`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty and must correspond to a registered bridge.
 - **Impact:** Selects the IBC client and counterparty configuration.
- `receiver`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty.
 - **Impact:** Destination address encoded in the cross-chain mint call.
- `amount`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be > 0 .
 - **Impact:** Tokens burned from the caller, and the amount is minted on the destination upon success.
- `timeoutTimestamp`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be $> \text{block.timestamp}$.
 - **Impact:** IBC packet time-out — on expiry, a refund is triggered via

onTimeoutPacket.

Branches and code coverage

Intended branches

- Tokens are burned.
 - ☒ Test coverage
- The GMP packet is sent.
 - ☒ Test coverage
- The pending transfer is stored.
 - ☒ Test coverage
- The IFTTransferInitiated event is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts with ReentrancyGuardReentrantCall if called reentrantly.
 - ☐ Negative test
- Reverts with IFTEmptyClientId if clientId is empty.
 - ☒ Negative test
- Reverts with IFTEmptyReceiver if receiver is empty.
 - ☒ Negative test
- Reverts with IFTZeroAmount if amount == 0.
 - ☒ Negative test
- Reverts with IFTTimeoutInPast if timeoutTimestamp <= block.timestamp.
 - ☒ Negative test
- Reverts with IFTBridgeNotFound if no bridge is registered for clientId.
 - ☒ Negative test
- Reverts (ERC-20 error) if the caller has insufficient balance.
 - ☐ Negative test

Function: `iftTransfer(string calldata clientId, string calldata receiver, uint256 amount)`

Convenience overload of [iftTransfer](#) that automatically sets `timeoutTimestamp` to `block.timestamp + 15 minutes`.

Inputs

- `clientId`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty and must correspond to a registered bridge.
 - **Impact:** Selects the IBC client and counterparty configuration.
- `receiver`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty.
 - **Impact:** Destination address is encoded in the cross-chain mint call.
- `amount`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be > 0 .
 - **Impact:** Tokens burned from the caller, and the amount is minted on the destination upon success.

Branches and code coverage

Intended branches

- The time-out is computed as `block.timestamp + 15 minutes`.
 - ☐ Test coverage
- Tokens are burned.
 - ☒ Test coverage
- The GMP packet is sent.
 - ☒ Test coverage
- The pending transfer is stored.
 - ☒ Test coverage
- The `IFTTransferInitiated` event is emitted.
 - ☐ Test coverage

Negative behavior

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test
- Reverts with `IFTEmptyClientId` if `clientId` is empty.
 - ☐ Negative test
- Reverts with `IFTEmptyReceiver` if `receiver` is empty.

- ☐ Negative test
- Reverts with IFTZeroAmount if amount == 0.
- ☐ Negative test
- Reverts with IFTBridgeNotFound if no bridge is registered for clientId.
- ☐ Negative test
- Reverts (ERC-20 error) if the caller has insufficient balance.
- ☐ Negative test

Function: `onAckPacket(bool success, IIBCAppCallbacks.OnAcknowledgementPacketCallback calldata msg_)`

This is an IBC sender callback invoked by the ICS27GMP contract after a GMP packet is acknowledged. On success, it deletes the pending transfer and emits IFTTransferCompleted. On failure, it mints the burned tokens back to the original sender.

Inputs

- success
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** Determines whether the transfer is finalized or refunded.
- msg_
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** Used to look up and clear the pending transfer record.

Branches and code coverage

Intended branches

- Success acknowledgment — the pending transfer is deleted.
 - ☒ Test coverage
- Success acknowledgment — IFTTransferCompleted is emitted.
 - ☒ Test coverage
- Error acknowledgment — tokens are minted back to sender.
 - ☒ Test coverage
- Error acknowledgment — the pending transfer is deleted from storage.

- ☒ Test coverage
- Error acknowledgment – IFTTransferRefunded is emitted.

☒ Test coverage

Negative behavior

- Reverts with ReentrancyGuardReentrantCall if called reentrantly.
- ☐ Negative test
- Reverts with IFTOnlyICS27GMP if msg.sender != _ics27Gmp.
- ☒ Negative test
- Reverts with IFTPendingTransferNotFound if no pending transfer exists for (sourceClient, sequence) (success path).
- ☒ Negative test
- Reverts with IFTPendingTransferNotFound if no pending transfer exists for (sourceClient, sequence) (failure/refund path).
- ☐ Negative test

Function: `onTimeoutPacket(IIBCAppCallbacks.OnTimeoutPacketCallback calldata msg_)`

This is an IBC sender callback invoked by the ICS27GMP contract when a GMP packet times out. It mints the burned tokens back to the original sender.

Inputs

- msg_
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** N/A.
 - **Impact:** Triggers the refund of burned tokens.

Branches and code coverage

Intended branches

- The pending transfer is found.
 - ☒ Test coverage
- Tokens are minted back to the sender.
 - ☒ Test coverage
- The pending transfer is deleted from storage.

☒ Test coverage

- IFTTransferRefunded is emitted.

☒ Test coverage

Negative behavior

- Reverts with ReentrancyGuardReentrantCall if called reentrantly.

☐ Negative test

- Reverts with IFTOnlyICS27GMP if msg.sender != _ics27Gmp.

☒ Negative test

- Reverts with IFTPendingTransferNotFound if no pending transfer exists.

☒ Negative test

Function: registerIFTBridge(string calldata clientId, string calldata counterpartyIFTAddress, address iftSendCallConstructor)

This function registers a new IFT bridge configuration for a given IBC client. The bridge defines the counterparty IFT contract address and a IFTSendCallConstructor used to construct the cross-chain mint call payload.

Inputs

- clientId
 - **Control:** Authority-controlled.
 - **Constraints:** Must be nonempty.
 - **Impact:** Key for the bridge mapping — determines which IBC client is used when initiating IFT transfers.
- counterpartyIFTAddress
 - **Control:** Authority-controlled.
 - **Constraints:** Must be nonempty.
 - **Impact:** The address of the IFT contract on the counterparty chain — used to authorize iftMint calls (compared against accountId.sender).
- iftSendCallConstructor
 - **Control:** Authority-controlled.
 - **Constraints:** Must be nonzero and must implement IFTSendCallConstructor (checked via ERC-165).
 - **Impact:** Constructs the ABI-encoded mint payload sent via ICS27GMP.

Branches and code coverage

Intended branches

- The bridge is stored in `_iftBridges[clientId]`.
 - ☒ Test coverage
- If a bridge for `clientId` already exists, it is silently overwritten (no existence check, no error).
 - ☐ Test coverage
- The `IFTBridgeRegistered` event is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts (authority check) if the caller is not the authority.
 - ☒ Negative test
- Reverts with `IFTEmptyClientId` if `clientId` is empty.
 - ☒ Negative test
- Reverts with `IFTEmptyCounterpartyAddress` if `counterpartyIFTAddress` is empty.
 - ☒ Negative test
- Reverts with `IFTZeroAddressConstructor` if `iftSendCallConstructor` is `address(0)`.
 - ☒ Negative test
- Reverts with `IFTInvalidConstructorInterface` if the constructor does not implement `IFTSendCallConstructor`.
 - ☒ Negative test

Function: `removeIFTBridge(string calldata clientId)`

This function removes an existing IFT bridge registration for the given client ID. After removal, `iftTransfer` for that client will revert.

Inputs

- `clientId`
 - **Control:** Authority-controlled.
 - **Constraints:** Must map to an existing registered bridge entry (`IFTBridgeNotFound` otherwise).
 - **Impact:** Deletes the bridge entry — any in-flight packets will still be resolved via callbacks, but new transfers are blocked.

Branches and code coverage

Intended branches

- The entry is deleted from `_iftBridges`.
 - ☒ Test coverage
- The `IFTBridgeRemoved` event is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts (authority check) if the caller is not the authority.
 - ☒ Negative test
- Reverts with `IFTBridgeNotFound` if no bridge is registered for `clientId`.
 - ☒ Negative test

5.8. Module: IFTOwnable.sol

Function: `iftMint(address receiver, uint256 amount)`

This function mints IFT tokens to a receiver address. It is called by an ICS27 interchain account (ICA) during cross-chain transfer reception. It authenticates the caller by verifying that it is a recognized ICA whose associated bridge configuration matches.

Inputs

- `receiver`
 - **Control:** ICS27Account-controlled (ultimately driven by the counterparty chain's GMP call).
 - **Constraints:** N/A (minting to address (0) would revert in ERC-20).
 - **Impact:** Address that receives the newly minted tokens.
- `amount`
 - **Control:** ICS27Account-controlled.
 - **Constraints:** N/A.
 - **Impact:** Amount of tokens minted.

Branches and code coverage

Intended branches

- The caller is a recognized ICA, the bridge matches, and salt is empty — tokens are

minted to the receiver.

☒ Test coverage

Negative behavior

- Reverts with `ICS27AccountNotFound` if `msg.sender` is not a recognized `ICS27Account`.

☒ Negative test

- Reverts with `IFTBridgeNotFound` if no bridge is registered for the ICA's `clientId`.

☒ Negative test

- Reverts with `IFTUnauthorizedMint` if the ICA's sender field does not match `bridge.counterpartyIFTAddress`.

☒ Negative test

- Reverts with `IFTUnexpectedSalt` if the ICA's salt is nonempty.

☒ Negative test

- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.

☐ Negative test

Function: `iftTransfer(string calldata clientId, string calldata receiver, uint256 amount, uint64 timeoutTimestamp)`

This function initiates a cross-chain fungible token transfer. It burns the specified amount from the sender, constructs a mint call via the configured `IIFTSendCallConstructor`, sends it as an `ICS27GMP` packet, and records the pending transfer for later acknowledgment/time-out handling. A second overload exists without `timeoutTimestamp` that uses a default 15-minute time-out.

Inputs

- `clientId`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty and must have a registered IFT bridge.
 - **Impact:** Determines which bridge configuration (counterparty address, call constructor) is used.
- `receiver`
 - **Control:** Caller-controlled.
 - **Constraints:** Must be nonempty. Further validated by the `IIFTSendCallConstructor.constructMintCall`.
 - **Impact:** The recipient address on the counterparty chain.
- `amount`

- **Control:** Caller-controlled.
- **Constraints:** Must be greater than zero. The sender must have sufficient balance (enforced by `_burn`).
- **Impact:** Amount of tokens burned locally and minted on the counterparty.
- `timeoutTimestamp`
 - **Control:** Caller-controlled (explicit overload) or default `block.timestamp + 15 min.`
 - **Constraints:** Must be greater than `block.timestamp`. Further validated by `ICS26Router.sendPacket` (within one day).
 - **Impact:** Controls the IBC packet time-out.

Branches and code coverage

Intended branches

- Tokens are burned, the mint call is constructed, the packet is sent, and the pending transfer is recorded.
 - ☒ Test coverage
- Default time-out overload uses `block.timestamp + 15 minutes`.
 - ☒ Test coverage

Negative behavior

- Reverts with `IFTEmptyClientId` if `clientId` is empty.
 - ☒ Negative test
- Reverts with `IFTEmptyReceiver` if the receiver is empty.
 - ☒ Negative test
- Reverts with `IFTZeroAmount` if the amount is 0.
 - ☒ Negative test
- Reverts with `IFTTimeoutInPast` if the time-out is in the past.
 - ☒ Negative test
- Reverts with `IFTBridgeNotFound` if the bridge is not found for `clientId`.
 - ☒ Negative test
- Reverts with `ERC20InsufficientBalance` if the sender has insufficient balance.
 - ☐ Negative test
- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test

Function call analysis

- `_burn(sender, amount)`
 - **What is controllable?** `amount`.
 - **If the return value is controllable, how is it used and how can it go wrong?** N/A — no return value. Decreases sender's balance.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `bridge.iftSendCallConstructor.constructMintCall(receiver, amount)`
 - **What is controllable?** `receiver` and `amount`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the calldata payload for the ICS27GMP packet. Not directly controllable beyond the inputs. For `EVMIFTSendCallConstructor`, it produces `abi.encodeCall(IIFT.iftMint, (addr, amount))`. For `CosmosIFTSendCallConstructor`, it produces `protojson MsgIFTMint`.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `$_ics27Gmp.sendCall(SendCallMsg)`
 - **What is controllable?** `timeoutTimestamp`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the sequence number — used as a key for the pending transfer record. Not controllable.
 - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `onAckPacket(bool success, IIBCAppCall-backs.OnAcknowledgementPacketCallback calldata msg_)`

This is a callback invoked by ICS27GMP (via `IBCSenderCallbacksLib`) when a cross-chain IFT transfer packet is acknowledged. On success, it deletes the pending transfer record. On failure, it refunds the tokens to the original sender by reminting.

Inputs

- `success`
 - **Control:** ICS27GMP-controlled (derived from acknowledgment comparison against universal error acknowledgment).
 - **Constraints:** N/A.
 - **Impact:** Determines whether the transfer succeeded or failed — `false` triggers a refund.
- `msg_.sourceClient`

- **Control:** ICS27GMP-controlled.
 - **Constraints:** Must correspond to an existing pending transfer.
 - **Impact:** Key for pending transfer lookup.
- `msg_.sequence`
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** Must correspond to an existing pending transfer with amount > 0.
 - **Impact:** Key for pending transfer lookup.

Branches and code coverage

Intended branches

- Success = true — the pending transfer is deleted, and `IFTTransferCompleted` is emitted.
 - ☒ Test coverage
- Success = false — tokens are reminted to the original sender, the pending transfer is deleted, and `IFTTransferRefunded` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts with `IFTOnlyICS27GMP` if the caller is not ICS27GMP.
 - ☒ Negative test
- Reverts with `IFTPendingTransferNotFound` if no pending transfer is found.
 - ☒ Negative test
- Reverts with `ReentrancyGuardReentrantCall` if called reentrantly.
 - ☐ Negative test

Function: `onTimeoutPacket(IIBCAppCallbacks.OnTimeoutPacketCallback calldata msg_)`

This is a callback invoked by ICS27GMP (via `IBCSenderCallbacksLib`) when a cross-chain IFT transfer packet times out. It refunds the originally burned tokens to the sender.

Inputs

- `msg_.sourceClient`
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** Must correspond to an existing pending transfer.

- **Impact:** Key for pending transfer lookup.
- msg_.sequence
 - **Control:** ICS27GMP-controlled.
 - **Constraints:** Must correspond to an existing pending transfer with amount > 0.
 - **Impact:** Key for pending transfer lookup.

Branches and code coverage

Intended branches

- Pending transfer is found — tokens are reminted to sender, and the record is deleted.
- ☒ Test coverage

Negative behavior

- Reverts with IFTOnlyICS27GMP if the caller is not ICS27GMP.
- ☒ Negative test
- Reverts with IFTPendingTransferNotFound if no pending transfer is found.
- ☒ Negative test
- Reverts with ReentrancyGuardReentrantCall if called reentrantly.
- ☐ Negative test

6. Assessment Results

During our assessment on the scoped IBC Solidity contracts, we discovered four findings. No critical issues were found. Two findings were of medium impact and two were of low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.